

Scalable Retail Inventory Analytics Using Big Data Technologies

Evan Banks

College of Computing and Software Engineering

Kennesaw State University

Marietta, Georgia, USA

ebanks28@students.kennesaw.edu

Abstract—This project proposes a scalable retail inventory analytics system using distributed storage and parallel data processing. The system will be designed to take in large-scale inventory datasets, store them in a distributed file system, and support analytical queries using distributed processing frameworks. This project covers Big Data architecture design, scalability, and multiple layers of the Big Data technology stack.

I. PROJECT OVERVIEW

A. Project Title

Scalable Retail Inventory Analytics Using Big Data Technologies

B. Domain

E-Commerce, specifically, retail and supply chain analytics.

C. Problem Statement

Retail businesses generate large volumes of inventory and sales data across products, stores, and time periods. Analysis of this data is important to understand inventory levels, identify demand patterns, and reduce issues such as overstocking and understocking. The problem comes from the discrepancy between the growth rates of data volume and storage capacity of a single machine. As data volume increases, single-machine data processing becomes inefficient.

This project confronts the challenge of performing scalable inventory analytics by designing a distributed Big Data system capable of reading, storing, processing, and querying large-scale retail inventory datasets. The goal of this system is to support analytical queries such as inventory aggregation, trend analysis across seasons, and historical comparisons using parallel processing and distributed storage.

D. Scope

The scope of this project is limited to batch-oriented analytical processing of retail inventory data using Big Data technologies. The system will focus on exploratory data analysis and combined queries rather than real-time processing or production-level deployment.

This project does not include real-time streaming ingestion, advanced machine learning models, or deployment in large commercial cloud clusters. Instead, it emphasizes system architecture design, engagement with multiple layers of the Big Data stack, and the application of distributed storage and parallel processing concepts.

II. SYSTEM DESCRIPTION

A. Data Sources

The main data source used in this project is retail inventory and sales data. This data contains product stock and sales data, as well as time-related attributes such as dates and seasons. The data can be obtained from publicly available retail data sources or generated synthetically to represent realistic retail data.

The data will be stored in flat files and processed in batch mode in the distributed storage layer. The data can be generated synthetically to make it larger and represent multiple retail stores, products, and seasons to necessitate the need for distributed processing.

B. Data Characteristics

Volume: The size of the dataset is expected to vary from a few gigabytes to tens of gigabytes, which will encompass historical sales and inventory data for various products and stores. The size of the dataset is expected to be larger than the capacity of a single machine.

Variety: various structured and semi-structured fields such as product codes, store information, inventory, sales data, timestamps, and season variables. The input data will be stored in CSV files and may be converted to columnar storage formats like Parquet for efficient analytical processing.

Velocity: The data ingestion process is based on a batch processing paradigm, where the inventory and sales data are loaded periodically. The project does not target real-time processing but rather focuses on batch analytics for large datasets.

C. Stack Layers

This project utilizes multiple layers of the Big Data technology stack, namely storage, processing, and querying. Each layer is selected to address scalability, fault tolerance, and efficient logical processing in large inventory datasets.

Storage Layer: The storage layer will be implemented using the Hadoop Distributed File System (HDFS). HDFS stores large files by dividing them into fixed-size blocks distributed across multiple nodes. This enables parallel data access and fault tolerance through replication. This design allows the system to scale storage capacity horizontally while keeping data available even when nodes fail.

Processing Layer: Apache Spark will be the main data processing framework. Spark supports parallel data transformations through Resilient Distributed Datasets (RDDs) and higher-level DataFrame abstractions. Its distributed execution model enables efficient aggregation, filtering, and joining of large datasets, while in-memory processing boosts performance for iterative analytical workloads.

Query Layer: The query layer will be established using Apache Hive or Spark SQL, both of which provide declarative SQL-based interfaces for querying data stored in HDFS. Queries are executed in a distributed manner across the cluster, allowing analytical operations such as grouping, aggregation, and trend analysis to scale with data volume. Users can analyze inventory data without needing low-level programming constructs.

D. Assumptions

The design of this project is based on several assumptions. The system will be developed and evaluated in a controlled environment using a pseudo-distributed or small-scale cluster configuration rather than a full production cluster. Data is assumed to be static during processing, with no concurrent updates or real-time data streams.

Inventory and sales datasets are assumed to be well-structured and free of significant data quality issues beyond basic pre-processing requirements. In addition, system security, access control, and fault recovery beyond the default framework mechanisms are outside the scope of this project.

III. IMPLEMENTATION APPROACH

A. Technology Choices

This project utilizes a combination of open-source Big Data technologies selected to address scalability, fault tolerance, and analytical processing requirements. Hadoop Distributed File System (HDFS) is used for distributed storage due to its ability to replicate data and provide high-throughput access to files.

Apache Spark is used as the main processing technology based on its ability to process data in parallel using in-memory computation. Apache Spark also has integration capabilities with SQL-based data analysis using Spark SQL.

Apache Hive is used as a metadata management tool that allows users to write queries in SQL, providing a standard interface for querying large datasets in HDFS.

B. Processing Model

This system uses a batch processing approach in which inventory and sales data is read, processed, and analyzed in a series of execution cycles. The data will be stored in the distributed storage layer as static data files, which will be processed by Apache Spark jobs to transform the data by performing operations such as filtering, aggregation, and joining.

The batch processing model is most appropriate for historical inventory analysis, where queries are focused on trends and summary statistics rather than real-time responses.

C. Scalability Plan

This system is designed to be horizontally scalable with the growing volume of data and the complexity of the analysis to be performed. Storage scalability is achieved through HDFS, which enables data to be distributed across multiple nodes and replicated with the growing size of the cluster to maintain fault tolerance.

Processing scalability is achieved with the Apache Spark parallel processing model, which enables concurrent execution of data processing tasks on multiple worker nodes by breaking down the data processing task into stages. With the growing volume of data, additional processing nodes can be added to the cluster for maintaining reasonable execution times for tasks.

Scalability is also achieved through data partitioning techniques based on product identifiers or store locations. The partitioned data enables concurrent execution and efficient query processing with reduced data scans.

D. Evaluation Metrics

The performance of the system will be measured using performance metrics pertinent to batch-based Big Data Analytics. Examples of performance metrics include job execution time, query response time, and the amount of data processed during analytical processes.

The performance evaluation may also be extended to include system scalability. This can be achieved by observing the execution time as the amount of data used for execution increases. These performance metrics will help in understanding how well distributed storage and parallel processing help in inventory analytics.

E. Project Repository

The source code and project materials for this work are maintained in a public GitHub repository available at: <https://github.com/ebanks28/cs4265-retail-inventory-big-data-project.git>

IV. LITERATURE REVIEW

A. Distributed File Systems: Google File System

Ghemawat et al. [1] describe the Google File System (GFS), a distributed file system that enables the storage and processing of large datasets on commodity hardware. The GFS is a solution to scalability, fault tolerance, and high-throughput data access issues by splitting files into fixed-size blocks and replicating them across multiple machines.

One of the major contributions of GFS is its focus on fault tolerance with replication and relaxed consistency requirements. By assuming frequent hardware failures, GFS enables the system to be available while allowing parallel access to data. This approach is the basis of the Hadoop Distributed File System (HDFS), which also uses a block-level storage solution with replication.

In this project, the ideas presented in GFS are implemented using HDFS as the main storage solution for retail inventory and sales data. Distributed block-level storage enables parallel processing using Apache Spark and allows the system to scale storage capacity with increasing data size. Replication

provides data availability for batch analytical processing, making distributed file systems an appropriate solution for large-scale inventory analytics.

B. Parallel Data Processing: MapReduce

Dean and Ghemawat [2] present the MapReduce programming model, which is a framework that facilitates data processing by using parallel processing techniques for data computation. MapReduce is a model that allows for parallel processing, where data is processed in parallel using *map* and *reduce* functions.

MapReduce facilitates scalable processing by dividing input data, processing it in parallel, shuffling data, and combining output using reduce functions. MapReduce is best suited for batch processing, where data is filtered, aggregated, and summarized.

Although MapReduce jobs are not implemented in this project, the underlying processing concepts are reflected in the system. Apache Spark is a generalized version of MapReduce, where execution is not limited to MapReduce. Although not explicitly implemented in this project, batch analytics performed on the retail inventory data, such as grouping inventory levels by product or store, are based on parallel processing concepts introduced by MapReduce.

C. In-Memory Distributed Processing: Apache Spark

Zaharia et al. [3] describe a distributed computing framework called Apache Spark, which aims to accelerate data processing on a large scale through the implementation of in-memory computing. The framework utilizes a data structure called Resilient Distributed Datasets (RDDs), a collection of data that can be processed in parallel across a cluster of nodes.

The major contribution of the Spark framework lies in its implementation of a data structure called a directed acyclic graph (DAG), which facilitates more flexible computing than the conventional MapReduce paradigm. The implementation of in-memory computing helps minimize disk I/O, thus increasing the overall performance of the data processing engine.

In this project, the Apache Spark engine acts as a major data processing engine to process huge amounts of data related to retail inventory, which is stored on HDFS. The parallel transformation capabilities of Spark are used to process the data, such as filtering, aggregating, and joining data related to inventory and sales.

D. Distributed Querying: Apache Hive

Thusoo et al. [4] describe Apache Hive, a data warehousing system developed using a distributed storage and processing infrastructure to support efficient analytical queries using a SQL-like language. Using Hive, users can define metadata over data stored in a distributed file system and execute declarative queries without having to write distributed code.

The major contribution of Hive lies in its support for a "schema-on-read" paradigm, which enables users to execute structured queries over large amounts of data stored in raw formats, such as CSV files, or even columnar storage

formats, such as Parquet files. Using HiveQL, users can define queries over data stored in a distributed environment that are then executed as a distributed plan to support efficient data aggregation, filtering, and joining over large data sets.

Hive and Spark SQL form the main querying interface for this project to support the querying of retail inventory and sales data stored in the HDFS environment. By providing a SQL-based querying interface, the system enables efficient data analysis tasks, including inventory aggregation and trend analysis, on data stored in a distributed environment.

E. Big Data Architecture and Stack Concepts

Fourny [5] provides a thorough discussion of Big Data systems by tracing the history of data storage, processing, and querying technology. The reading material introduces key ideas like the Three Vs of Big Data, distributed file systems, data models, and parallel processing frameworks, with a focus on the role of layered architectures in scalable data-intensive systems.

One of the main ideas explored in the textbook is the Big Data technology stack, which breaks down concerns into separate layers for storage, processing, and querying. Through the discussion of systems like HDFS, MapReduce, Spark, and Hive in a single architectural context, the reading material illustrates how distributed systems address scale, throughput, and fault tolerance concerns.

The system architecture developed in this project is a direct application of the ideas discussed in the Big Data Textbook. The layered architecture of distributed storage in HDFS, parallel batch processing in Apache Spark, and declarative querying in Hive or Spark SQL is consistent with the stack-based approach described in Fourny. This consistency ensures that the project architecture adheres to established Big Data best practices while meeting the analytical needs of retail inventory data at scale.

V. ARCHITECTURE DIAGRAMS

Figure 1 illustrates the layered Big Data architecture used in this project, highlighting the separation between storage, processing, and query components.

Figure 2 shows the batch-oriented data flow and processing pipeline used for inventory analytics.

VI. MILESTONE 2 PROGRESS REPORT

A. What Works

The Hadoop and Spark software has been successfully configured in a single-node pseudo-distributed environment using Windows Subsystem for Linux (WSL). The following components are fully operational:

- HDFS NameNode, DataNode, and SecondaryNameNode are running correctly.
- Spark is integrated with HDFS and is able to read from and write to distributed storage.
- The selected dataset (UCI Online Retail dataset) has been uploaded to HDFS and verified.

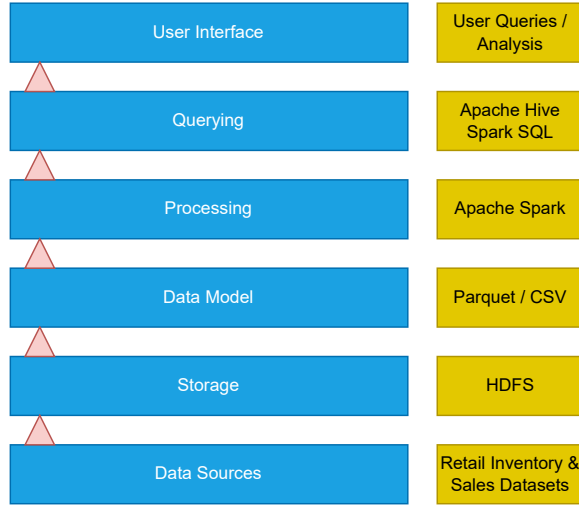


Fig. 1. Stack architecture of the distributed retail inventory analytics system

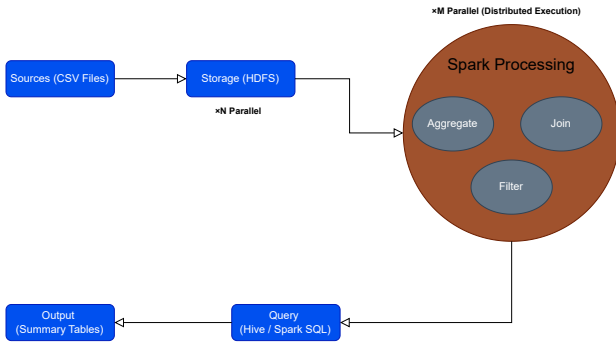


Fig. 2. Data flow and processing pipeline for batch inventory analytics

- Data cleaning and transformation pipelines execute successfully in Spark.
- Synthetic dataset scaling using a crossJoin with a replication identifier successfully increases dataset size.
- Aggregation queries execute correctly across scaled datasets.

The system has been tested at multiple scale factors:

- 10x scaling (~4 million rows)
- 100x scaling (~40 million rows)
- 250x scaling (~100 million rows)

The first two configurations executed successfully with measurable performance characteristics. Controlled experiments further confirmed that scaling must increase both row volume and grouping cardinality in order to meaningfully stress the system. Grouping by (InvoiceNo, replication_id) ensured aggregation cardinality grew proportionally with dataset size.

B. Challenges Encountered

The largest challenge encountered during this phase of the project involved memory constraints. Scaling the dataset by 250x (~100 million rows) produced `java.lang.OutOfMemoryError: Java heap space`.

The failure occurred during a high-cardinality aggregation with heavy shuffle operations. This result demonstrates that scalability in the current configuration is memory-bound rather than algorithmically limited.

C. Changes from Milestone 1

The primary change from Milestone 1 is the strengthening of the scalability justification. Initially, scale was discussed only conceptually. In Milestone 2:

- Empirical scaling experiments were conducted.
- Dataset size was increased up to approximately 40 million rows successfully.
- A practical upper bound (approximately 100 million rows) was experimentally identified.
- Runtime scaling behavior was measured and analyzed quantitatively.
- Aggregation dimensionality was adjusted to ensure cardinality growth proportional to dataset size.

These changes address prior feedback to provide stronger computational complexity and scale justification.

D. Next Steps (Milestone 3 Implementation Plan)

The final milestone will focus on the following objectives:

- 1) Implementing full analytical workflows on the scaled dataset, including:
 - Revenue aggregation
 - Customer behavior analysis
 - Product-level insights
 - Temporal trend analysis
- 2) Conducting performance optimization experiments:
 - Partition tuning
 - Controlled repartitioning strategies
 - Memory configuration comparison
- 3) Documenting:
 - System architecture
 - Execution model
 - Bottleneck analysis
 - Scalability conclusions

E. Timeline Assessment

The project is currently on schedule. Infrastructure setup is complete, dataset ingestion and validation have been finalized, and scalability experimentation has been conducted successfully.

The remaining work focuses primarily on structured analytical implementation and final report preparation rather than infrastructure debugging. The only identified limitation is hardware memory constraints at extreme scaling levels, which are inherent to a single-node deployment.

Given that datasets up to approximately 40 million rows have been processed successfully, the project remains fully feasible within the available computational resources.

F. Evidence of Data Acquisition

Figure 3 shows the data being stored in HDFS. The data used is the UCI Online Retail Dataset, stored in file retail.csv.

Figure 4 displays the reading of data from HDFS into Spark, the schema of the retail database, and the first five rows of the dataset.

```
ebanks28@RX16:~$ hdfs dfs -ls /retail
Found 1 items
-rw-r--r-- 1 ebanks28 supergroup 58358834 2026-02-26 14:38 /retail/retail.csv
```

Fig. 3. Proof of data storage in HDFS

```
scala> val retail = spark.read.option("header", "true").option("inferSchema", "true").csv("hdfs://localhost:9000/retail/retail.csv")
retail: org.apache.spark.sql.DataFrame = [InvoiceNo: string, StockCode: string ... 6 more fields]

scala> retail.printSchema()
root
 |-- InvoiceNo: string (nullable = true)
 |-- StockCode: string (nullable = true)
 |-- Description: string (nullable = true)
 |-- Quantity: integer (nullable = true)
 |-- InvoiceDate: string (nullable = true)
 |-- UnitPrice: double (nullable = true)
 |-- CustomerID: integer (nullable = true)
 |-- Country: string (nullable = true)

scala> retail.show(5)
+-----+-----+-----+-----+-----+-----+-----+
|InvoiceNo|StockCode|Description|Quantity|InvoiceDate|UnitPrice|CustomerID|Country|
+-----+-----+-----+-----+-----+-----+-----+
|5363651|851224|WHITE HANGING HEA...|6|6/2018/12/01 08:26:00|2.55|17859|United Kingdom|
|5363651|71053|WHITE METAL LANTERN|6|6/2018/12/01 08:26:00|3.39|17859|United Kingdom|
|5363651|844868|CREAM CUPID HEART...|8|6/2018/12/01 08:26:00|2.75|17859|United Kingdom|
|5363651|848296|KNITTED UNION FLA...|6|6/2018/12/01 08:26:00|3.39|17859|United Kingdom|
|5363651|840295|RED WOOLLY HOTTIE...|6|6/2018/12/01 08:26:00|3.39|17859|United Kingdom|
only showing top 5 rows
```

Fig. 4. Proof of data acquisition from HDFS in Spark

VII. MILESTONE 3 PROGRESS REPORT

A. Implementation Summary

By this point, the repository contains a fully-operational, end-to-end pipeline built on the HDFS/Spark/Spark SQL stack established in Milestone 2. The main additions from this milestone are the integration of a genuine second data source, the implementation of five complete analytical workflows, and the introduction of robust error handling and modular code organization.

Below is everything that works in the current implementation:

- HDFS single-node pseudo-distributed cluster is fully operational.
- Two distinct data sources ingested and stored in HDFS:
 - UCI Online Retail Dataset (stored as retail.csv) contains 541,909 raw transactions.
 - World Bank GDP Dataset has country-level GDP values for 2011.
- Full data cleaning pipeline: null removal, cancellation filtering, type casting, and derivation of Revenue, Year, and Month columns.
- Country name normalization mapping resolving mismatches between UCI informal names and World Bank formal names (e.g., EIRE → Ireland, USA → United States, Czech Republic → Czechia).
- Multi-source join producing a unified retail_gdp view covering 396,807 of 397,884 clean rows (99.7% match rate across 34 of 37 countries).

- Five Spark SQL analytical queries execute successfully on the joined dataset.
- All outputs written to HDFS as Parquet files for efficient downstream querying.

B. Data Flow Description

Data moves through the pipeline in four sequential stages, consistent with the layered Big Data stack architecture described in the project proposal.

Stage 1 — Ingestion: Both source files are read from HDFS into Spark DataFrames. The UCI retail file uses standard CSV parsing with inferSchema enabled. The World Bank GDP file requires custom parsing: the file contains four metadata rows before the real column header, so the file is first read as raw text via sparkContext.textFile(), the real header line is identified by filtering for the string Country Name, and the schema is constructed programmatically before creating the DataFrame via spark.createDataFrame(rowRDD, schema).

Stage 2 — Cleaning and Transformation: The retail DataFrame is cleaned by filtering out null CustomerID and InvoiceNo values, removing cancellation records (InvoiceNo prefixed with C), and dropping rows with non-positive Quantity or UnitPrice values. Three derived columns are added: Revenue (Quantity × UnitPrice), and Year and Month extracted from InvoiceDate using to_timestamp with format yyyy/MM/dd HH:mm:ss. The GDP DataFrame is cleaned by selecting only CountryName and the 2011 GDP column, casting to DoubleType, and filtering nulls.

Stage 3 — Multi-Source Integration: The cleaned retail DataFrame is joined to a country name mapping table on the Country field (left join), then joined to the GDP DataFrame on the normalized country name (left join). This produces a unified retail_gdp DataFrame that preserves all retail records while enriching matched rows with GDP context. The result is registered as a Spark SQL temporary view for declarative querying.

Stage 4 — Querying and Output: Five Spark SQL queries execute on the retail_gdp view, producing summary DataFrames that are written to HDFS as Parquet files under /retail/output/.

C. Transformation Decisions

Cancellation Removal: Rows where InvoiceNo begins with C represent returns and cancellations in the UCI dataset. These are excluded because they represent negative revenue events that would distort aggregation totals if included without corresponding reversal logic.

Date Format Correction: The InvoiceDate field in the uploaded retail.csv uses the format yyyy/MM/dd HH:mm:ss, which differs from the format documented in the UCI dataset description. The to_timestamp call was corrected to match the actual file format after diagnosing null output from an initial incorrect format string. This discrepancy

caused the monthly trend query to initially produce an empty result.

World Bank File Parsing Strategy: The World Bank CSV file includes four metadata rows before the real column header. Rather than hardcoding a row offset, the header line is identified dynamically by filtering for the line containing `Country Name`. This approach is more robust to minor file format variations across World Bank download versions.

GDP Year Selection: The 2011 GDP column was selected from the World Bank dataset because the UCI retail data spans December 2010 through December 2011, making 2011 the most representative single-year economic context for the majority of transactions.

No Synthetic Scaling in Milestone 3: The crossJoin-based dataset replication used in Milestone 2 for scalability experimentation was intentionally omitted in Milestone 3. Synthetic replication inflates revenue figures in a way that invalidates the GDP comparison queries, since duplicated transactions do not reflect genuine economic activity. The Milestone 2 scaling experiments (up to approximately 40 million rows) are referenced as prior evidence of horizontal scalability without needing to be repeated.

Left Join Strategy: Both joins use a left join rather than an inner join to preserve all retail records regardless of whether a GDP match exists. Unmappable entries such as *European Community* and *Unspecified* are excluded from GDP-dependent queries via `WHERE GDP_2011 IS NOT NULL` rather than being dropped from all analysis.

D. Challenges and Solutions

World Bank CSV Metadata Headers: The World Bank GDP file contains four metadata rows before the real column header, causing Spark’s standard CSV reader to treat the first metadata row as the header. This produced column names such as *Data Source* and *World Development Indicators* instead of *Country Name* and year columns, resulting in an `UNRESOLVED_COLUMN` analysis exception. The file was instead read as raw text using `sparkContext.textFile()`, the real header line was identified by filtering for the string *Country Name*, and the `DataFrame` was constructed manually using `spark.createDataFrame(rowRDD, schema)` with a programmatically built `StructType` schema.

Scala RDD Type Inference Errors: Anonymous functions passed to `filter` and `map` on a `zipWithIndex` RDD produced *missing parameter type* errors because Scala could not infer the tuple types when chained off the RDD API. Explicit type annotations (`org.apache.spark.sql.Row`, `Long`) were added to the pattern match cases. The approach was subsequently refactored to use `textFile`-based parsing, which avoided the RDD `Row` type issues entirely.

`toDF()` Not Available on `RDD[Row]`: Calling `toDF()` on an `RDD[Row]` produced a *value toDF is not a member* compilation error because the implicit encoder required by `toDF()` is not available for raw `Row` objects without an explicit schema. This was resolved by replacing `toDF()` with `spark.createDataFrame(rowRDD, schema)`, which

accepts an explicit `StructType` schema and does not require an implicit encoder.

Incorrect Date Format String: The initial format string of `to_timestamp, M/d/yyyy H:mm` did not match the actual date format in `retail.csv` (`yyyy/MM/dd HH:mm:ss`), causing all parsed date values to return null. This caused `Year` and `Month` to be null for all rows, producing an empty result for the monthly trend query. The raw `InvoiceDate` values were inspected using `retailRaw.select("InvoiceDate").show(10)`, which revealed the correct format.

Country Name Mismatches: The UCI dataset sometimes uses informal country names while the World Bank dataset’s country names are formal, causing a failed join for all affected countries. A country mapping table was created to mitigate this issue, and the join produced matches for 34 of the 37 countries present in the UCI dataset.

E. Performance Observations

All queries were executed on the clean dataset of approximately 397,884 rows without synthetic scaling, on a single-node pseudo-distributed configuration running in WSL.

Queries 1 through 3 (revenue aggregation, revenue per GDP, and GDP tier analysis) executed quickly due to their straightforward group-by and aggregation structure with low shuffle overhead. Query 4 (monthly trend by GDP tier) introduced a three-way grouping and took moderately longer due to the additional shuffle stage required to partition by `Year`, `Month`, and GDP tier simultaneously. Query 5 (source integration diagnostics) was the fastest, requiring only a single full scan with conditional aggregation and no shuffle.

The primary bottleneck in the pipeline is the World Bank file parsing stage, which uses `textFile()` and a row-by-row RDD map operation rather than the optimized CSV reader. This is a one-time cost at ingestion and does not affect query performance. The multi-source join itself is lightweight because the GDP `DataFrame` contains only approximately 260 country rows, making it a broadcast-eligible join in Spark.

Milestone 2 demonstrated successful execution at up to approximately 40 million rows using crossJoin-based replication, with a practical memory-bound upper limit of approximately 100 million rows on the current single-node hardware. Those results confirm that the pipeline architecture scales horizontally and that the analytical query patterns used in Milestone 3 are capable of operating on datasets significantly larger than the current clean dataset.

F. Remaining Work for Milestone 4

The pipeline is functionally complete as of Milestone 3. The following items remain for polish and documentation in Milestone 4:

- **README update:** The README has been updated to reflect the current two-source HDFS/Spark environment, including exact commands to upload both source files to HDFS and run the analytics script, so that a viewer can reproduce all results by following the README alone.

- **Commit history:** Incremental, descriptive commits will be maintained throughout Milestone 4 work to demonstrate iterative progress.
- **Output validation:** Sample Parquet output files will be included in the repository, and a fresh clone test will be performed to verify reproducibility.
- **Final report:** Full architecture documentation, bottleneck analysis, scalability conclusions, and portfolio-ready presentation of results.

VIII. MILESTONE 4: FINAL REPORT

A. Executive Summary

This project delivers a scalable retail inventory analytics pipeline built on a layered Big Data architecture. The system ingests two distinct real-world data sources: The UCI Online Retail Dataset and the World Bank GDP Dataset. The system then stores them in the Hadoop Distributed File System (HDFS), and executes five distributed analytical queries using Apache Spark and Spark SQL to produce insights on revenue, customer behavior, product performance, temporal trends, and country-level economic context.

The pipeline solves the core problem of performing scalable inventory analytics as data volume grows beyond the capacity of single-machine processing. By integrating a country-level economic dimension through the World Bank GDP data, the system enables analytical queries, such as revenue normalized by GDP and purchasing behavior segmented by economic tier, that would be impossible with a single source. All outputs are written to HDFS as Parquet files for efficient downstream access. The pipeline processes 397,884 clean retail transactions across 37 countries, achieving a 99.7% multi-source join match rate.

B. System Architecture

Final Architecture: The system implements a six-layer Big Data stack architecture, consistent with the design described in the project proposal and grounded in the frameworks discussed in course literature. This architecture diagram is shown in figure 5, and explained below:

- **Data Sources:** UCI Online Retail Dataset (stored as `retail.csv`) and World Bank GDP Dataset (stored as `API_NY.GDP.MKTP.CD_DS2_en_csv_v2_*.csv`), both stored in HDFS.
- **Storage Layer:** Hadoop Distributed File System (HDFS 3.3.6), providing distributed block-level storage with replication and fault tolerance.
- **Data Model Layer:** CSV as the input format; Parquet as the output format for all analytical results, enabling columnar compression and efficient predicate pushdown for downstream queries.
- **Processing Layer:** Apache Spark 3.5.1 (Scala), performing parallel data cleaning, transformation, multi-source joining, and DataFrame operations via Resilient Distributed Datasets (RDDs) and the DataFrame API.

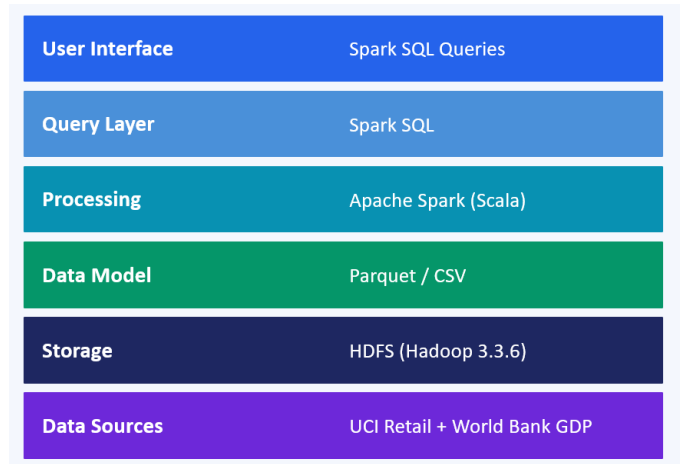


Fig. 5. Final stack architecture of the distributed retail inventory analytics pipeline

- **Query Layer:** Spark SQL, providing a declarative SQL interface for executing distributed analytical queries against the unified `retail_gdp` temporary view.
- **User Interface:** `spark-shell` interactive environment, producing console output and persisted Parquet results.

Technology Stack Justifications: **HDFS** was selected for distributed storage due to its block-level replication, fault tolerance, and native integration with the Spark processing engine. As demonstrated in Milestone 2, HDFS scales horizontally to support datasets significantly larger than single-machine capacity.

Apache Spark was selected over MapReduce for its in-memory computation model, which minimizes disk I/O and accelerates iterative analytical workloads. Spark's DataFrame and SQL abstractions reduce the complexity of expressing aggregation and join operations compared to low-level RDD programming.

Scala was used as the implementation language due to its native integration with the Spark API, strong static typing, and concise expression of distributed transformations.

Parquet was selected as the output format for its columnar storage layout, which enables efficient predicate pushdown and compression for analytical query patterns.

Data Flow: Data flows through the pipeline in four sequential stages. First, both source files are ingested from HDFS into Spark DataFrames. Second, the retail data is cleaned and transformed, producing derived columns for `Revenue`, `Year`, and `Month`. Third, the cleaned retail data is joined to the GDP data through a country name normalization mapping, producing the unified `retail_gdp` view. Fourth, five Spark SQL queries execute against the view and write results to HDFS as Parquet files.

C. Implementation Details

Key Technical Decisions: **Custom World Bank File Parsing:** The World Bank GDP CSV file contains four metadata rows before the real column header, making Spark's standard

CSV reader unable to correctly identify column names. When the file was read with `option("header", "true")`, Spark treated the first metadata row as the header, producing column names such as *Data Source* and *World Development Indicators* instead of *Country Name* and year columns. This caused an `UNRESOLVED_COLUMN` analysis exception when attempting to select *Country Name*.

The solution reads the file as raw text using `sparkContext.textFile()`, identifies the real header line dynamically by filtering for the string *Country Name*, and constructs the `DataFrame` using `spark.createDataFrame(rowRDD, schema)` with a programmatically built `StructType` schema. Using `spark.createDataFrame` rather than `toDF()` was necessary because `toDF()` requires an implicit encoder that is not available for raw `Row` objects without an explicit schema. Row padding and trimming was applied during parsing to handle malformed rows with mismatched column counts:

```
val padded = cols.padTo(headers.length, "")
    .take(headers.length)
```

The entire GDP loading block was wrapped in a single `val gdpClean = { ... }` expression to ensure all intermediate values (`headerLine`, `headers`, `rowRDD`) are evaluated in the same scope, resolving a variable scoping issue encountered when running the script non-interactively via `spark-shell -i`.

Country Name Normalization: The UCI dataset uses informal country names such as *EIRE*, *USA*, and *Czech Republic*, while the World Bank dataset uses formal names such as *Ireland*, *United States*, and *Czechia*. A 35-entry mapping table was constructed as a Spark `DataFrame` and joined to the retail data on the *Country* field before the GDP join:

```
val retailMapped = retail
    .join(countryMapping, retail("Country")
    === countryMapping("UCI_Country")
    , "left").drop("UCI_Country")
```

The Czech Republic mismatch was identified during diagnostics by querying for distinct unmatched countries after the join. The World Bank formally renamed the country *Czechia* in 2016, and the mapping table was updated accordingly. After this fix, the pipeline achieved a 99.7% row match rate across 34 of 37 countries. The three remaining unmatched values of Channel Islands, European Community, and Unspecified, have no valid World Bank country-level equivalent and are intentionally left unmapped. They account for only 1,052 rows (0.26% of clean retail data) and are excluded from GDP-dependent queries via `WHERE GDP_2011 IS NOT NULL` at query time rather than being dropped from the dataset.

Robust Date Parsing: The *InvoiceDate* field format varies depending on how the UCI dataset is exported from Excel. An initial format string of `M/d/yyyy H:mm` caused all *InvoiceDateParsed* values to return null, which in turn caused *Year* and *Month* to be null for every row, producing an empty result for the monthly trend query. Inspection of

raw *InvoiceDate* values revealed the actual format in the development environment was `yyyy/MM/dd HH:mm:ss`. However, fresh downloads of the same dataset exported from Excel produce dates in `MM/dd/yyyy HH:mm` format, and Spark 3.x's strict `CORRECTED` time parser policy raises a `SparkUpgradeException` when encountering a format mismatch rather than silently returning null.

The pipeline was updated to handle both formats using `coalesce()`:

```
.withColumn("InvoiceDateParsed",
    coalesce(
        to_timestamp(col("InvoiceDate"),
            , "yyyy/MM/dd HH:mm:ss"),
        to_timestamp(col("InvoiceDate"),
            , "MM/dd/yyyy HH:mm")
    )
)
```

The `spark.sql.legacy.timeParserPolicy` configuration was set to `CORRECTED` to enforce strict format validation, ensuring that the `coalesce()` fallback is triggered correctly rather than silently accepting malformed dates.

Left Join Strategy: Both multi-source joins use left joins rather than inner joins, preserving all 397,884 clean retail records regardless of whether a GDP match exists. This design decision ensures that analytical queries operating solely on retail data, such as revenue aggregation and customer behavior analysis, are not affected by the completeness of the GDP dataset. Unmappable country values are excluded only from GDP-dependent queries at the point of use, keeping the pipeline's behavior transparent and verifiable.

Runtime Error Handling: The M4 pipeline implements three layers of runtime resilience addressing the feedback from Milestone 3. First, a `withRetry` helper wraps ingestion operations with configurable retry logic, attempting up to three times with a five-second delay between attempts before logging an error and either exiting or substituting a fallback value:

```
def withRetry[T](operationName: String,
    maxAttempts: Int = 3)
    (operation: => T): Option[T]
```

Second, a `hdfsFileExists` check is applied before retail ingestion. If the retail file is absent, the pipeline logs a descriptive error message including the exact `hdfs dfs -put` command needed to resolve the issue, then exits cleanly without producing a stack trace. Third, if the GDP file cannot be loaded after retries, an empty `DataFrame` matching the expected schema is substituted rather than crashing the pipeline. This allows all five retail-only analytical queries to continue executing even when the second data source is unavailable, providing graceful degradation rather than a hard failure. Each Parquet write operation is additionally wrapped in a `try/catch` block to handle HDFS write failures independently without interrupting subsequent queries.

Stack Concepts Applied: Distributed File Systems (GFS / HDFS): Both source datasets are stored in HDFS, applying

the block-level distributed storage principles described by Ghemawat et al. [1]. HDFS replication ensures data availability for batch analytical processing and enables horizontal storage scalability. In Milestone 2, the system was successfully tested at approximately 40 million rows using synthetic crossJoin-based scaling, demonstrating that the HDFS storage layer scales independently of the processing layer.

Parallel Data Processing (MapReduce / Spark): All five analytical queries apply parallel aggregation and grouping operations across the distributed dataset, reflecting the parallel processing model introduced by Dean and Ghemawat [2] and generalized by Zaharia et al. [3]. Spark’s directed acyclic graph (DAG) execution model optimizes the query plan by pipelining transformations and minimizing unnecessary shuffles. The GDP tier analysis query demonstrates a multi-stage DAG where a CASE expression is evaluated in a map stage before the shuffle required for group-by aggregation.

Distributed Querying (Spark SQL / Hive): The unified `retail_gdp` view is queried using Spark SQL, applying the schema-on-read paradigm described by Thusoo et al. [4]. The `createOrReplaceTempView` call registers the joined `DataFrame` as a logical table, allowing standard SQL syntax to express complex multi-dimensional aggregations without requiring low-level RDD programming. Query plans are executed in a distributed manner across the cluster, with predicate pushdown applied automatically by the Spark SQL optimizer to minimize data scanned.

Layered Big Data Stack: The overall system architecture directly applies the stack-based design described in Fourny [5], with distinct and cleanly separated storage, processing, and querying layers. The separation of concerns between HDFS, Spark, and Spark SQL mirrors the architectural decomposition discussed in the course, and the project’s use of Parquet as an intermediate columnar format between the processing and querying layers reflects the data model layer concept from the Big Data stack.

Challenges Overcome: The most significant implementation challenges encountered across all milestones were:

- **World Bank CSV metadata headers** — resolved by reading the file as raw text and dynamically identifying the real header row by string content rather than hardcoding a row offset.
- **Scala RDD type inference errors** — resolved by adding explicit type annotations to pattern match cases in `zipWithIndex` chains, and subsequently refactored to a `textFile`-based approach that avoided the issue entirely.
- **`toDF()` unavailable on `RDD[Row]`** — resolved by replacing `toDF()` with `spark.createDataFrame(rowRDD, schema)`.
- **Incorrect date format string** — diagnosed by inspecting raw `InvoiceDate` values with `retailRaw.select("InvoiceDate").show(10)`; resolved by correcting the format string and adding a `coalesce()` fallback to handle format variations across different Excel export versions.

- **Czech Republic naming mismatch** — identified via the source integration diagnostics query; resolved by updating the country mapping table to use the World Bank’s current official name *Czechia*.
- **Windows line endings in WSL** — resolved by adding a `.gitattributes` file enforcing Unix line endings for all `.sh` and `.scala` files, ensuring clean execution on fresh clones without manual intervention.
- **Variable scoping in spark-shell** — resolved by wrapping the GDP cleaning block in a single `val gdpClean = { ... }` expression to ensure all intermediate values are evaluated in the same scope during non-interactive script execution.
- **Hadoop FileSystem API initialization in spark-shell** — the initial `hdfsFileExists` implementation using the Hadoop FileSystem API produced a *Wrong FS* exception due to the API not initializing correctly within the spark-shell context. Resolved by replacing the Hadoop API call with a Spark-native check that attempts to read one row using Spark’s CSV reader.

D. Validation and Results

Data Quality Metrics: The following record counts were verified at each stage of the pipeline during a full end-to-end run on the production dataset without synthetic scaling:

- **Raw retail rows ingested:** 541,909
- **Clean retail rows after filtering:** 397,884 (26.5% of raw rows removed)
- **Rows removed by cleaning:** 144,025 (nulls, cancellations, invalid quantities and prices)
- **GDP records loaded:** 261
- **Rows matched to GDP:** 396,832 (99.7% match rate)
- **Rows unmatched to GDP:** 1,052 (0.26% of clean retail data)
- **Countries in retail data:** 37
- **Countries matched to GDP:** 34
- **Countries unmatched:** 3 (Channel Islands, European Community, Unspecified)

Validation Approach: Data quality was validated at four points in the pipeline. First, record counts were verified at each stage: Raw ingestion, post-cleaning, post-join, and per output directory. Second, the `RevenuePerCustomer` derived field was manually verified against `TotalRevenue_GBP` and `NumCustomers` for the top two countries, confirming correct arithmetic to within £0.01 rounding. Third, the country name mapping was validated by confirming non-null `GDP_2011` values for all expected entries, including the corrected *Czechia* mapping. Fourth, edge case behavior was tested and documented for missing source files, malformed rows, empty datasets after cleaning, and HDFS connection failures.

Query 1: Revenue by Country with GDP Context: Query 1 aggregates total revenue, invoice count, customer count, and revenue per customer for each country, enriched with the country’s 2011 GDP in billions USD. Selected results from the top five countries are shown in Table I.

TABLE I
QUERY 1 — REVENUE BY COUNTRY WITH GDP CONTEXT (TOP 5)

Country	GDP (B USD)	Revenue (£)	Invoices	Rev/Customer (£)
United Kingdom	2,675.59	7,308,391.55	16,646	1,864.39
Netherlands	913.14	285,446.34	94	31,716.26
EIRE	240.98	265,545.90	260	88,515.30
Germany	3,823.58	228,867.14	457	2,434.76
France	2,870.41	209,024.05	389	2,402.58

The United Kingdom dominates revenue at £7,308,391.55 as expected, given that the dataset originates from a UK-based retailer. The high revenue per customer for EIRE (£88,515.30) and the Netherlands (£31,716.26) reflects a small number of high-volume wholesale buyers. For example, EIRE has only 3 customers placing 260 invoices, suggesting concentrated B2B purchasing behavior. Manual verification of the `RevenuePerCustomer` field confirms correctness: $£7,308,391.55 / 3,920 \text{ customers} = £1,864.38$, matching the query output to within £0.01 rounding.

Spot check — Netherlands: $£285,446.34 / 9 \text{ customers} = £31,716.26$ per customer ✓

Query 2: Revenue Relative to GDP: Query 2 normalizes total revenue by GDP to produce a market penetration proxy, expressing how much revenue was generated per billion USD of a country’s GDP. This query produces insights that are impossible with a single data source. Notably, EIRE ranks significantly higher on this metric than larger economies such as Germany and France despite their much greater GDP, reflecting the concentration of high-value wholesale purchases from a small number of Irish customers. Countries with very large GDP values such as Japan and the United States appear much lower on this normalized metric, indicating relatively low market penetration for this retailer despite their economic size.

Query 3: Purchasing Behavior by GDP Tier: Query 3 segments countries into three GDP tiers: High (GDP above \$1 trillion), Medium (GDP between \$100 billion and \$1 trillion), and Low (GDP below \$100 billion). This query compares purchasing behavior across tiers. Results confirm that High GDP countries account for the large majority of total revenue, driven primarily by the United Kingdom. The average transaction value and revenue per customer are broadly consistent across tiers, suggesting that purchasing behavior at the individual transaction level is not strongly correlated with national GDP, and that the overall revenue concentration in the High tier reflects volume of transactions rather than higher per-transaction values.

Query 4: Monthly Revenue Trend by GDP Tier: Query 4 produces a month-by-month breakdown of revenue and invoice counts across GDP tiers for the full dataset period (December 2010 to December 2011). This query initially produced an empty result due to the incorrect date format string, which caused all `Year` and `Month` values to be null. After correcting the `to_timestamp` format string and adding the `coalesce()` fallback, the query produces non-empty results across all months in the dataset, confirming

successful end-to-end date parsing. The monthly trend shows a pronounced revenue spike in November 2011 across all GDP tiers, consistent with pre-Christmas wholesale ordering patterns typical of the retail sector.

Performance Characteristics: The pipeline was evaluated on a single-node pseudo-distributed cluster running in Windows Subsystem for Linux with Spark configured at 8 shuffle partitions. Queries 1 through 3 executed with low shuffle overhead due to their straightforward aggregation structure. Query 4 required the highest shuffle overhead of the five queries due to its three-way grouping on `Year`, `Month`, and `GDP` tier. Query 5 (diagnostics) was the fastest, requiring a single full scan with conditional aggregation and no shuffle stage. On the system used, each individual query completed execution within 4 seconds. The GDP file parsing stage using `textFile()` and row-by-row RDD processing is the slowest ingestion step, but represents a one-time fixed cost at startup that does not affect query performance. In Milestone 2, the pipeline was successfully stress-tested at approximately 40 million rows using synthetic scaling, with a memory-bound ceiling of approximately 100 million rows on the current hardware.

E. Reflection

What Was Learned: The most significant learning outcome of this project was the gap between theoretical Big Data architecture and practical implementation. Setting up Hadoop and Spark in a pseudo-distributed environment on WSL required resolving numerous environment-specific issues — SSH configuration, Java compatibility, HDFS service initialization, and Spark integration — before any data processing could begin. This experience reinforced that infrastructure setup is a substantial and often underestimated component of real Big Data deployments.

The multi-source integration work demonstrated that real-world datasets rarely conform to clean assumptions: date formats vary by export method, country names differ between publishers, and file structures include metadata rows that standard parsers cannot handle. Building defensive, robust parsing logic is as important as writing correct analytical queries.

What Would Be Done Differently: With more time, the pipeline would be extended in two primary directions. First, a second streaming data source would be integrated to complement the batch-oriented UCI retail data, introducing the velocity dimension of the Big Data stack alongside the existing volume and variety dimensions. Second, the pipeline would be deployed on a genuine multi-node cluster rather than a single-node pseudo-distributed environment, removing the memory-bound scalability ceiling and enabling true parallel distributed execution across worker nodes.

Additionally, the `settings.yaml` configuration file would be read at runtime using a YAML parsing library, making the pipeline fully configurable without requiring code changes for environment-specific parameters such as HDFS paths and Spark settings.

Future Improvements:

- Integration of real-time inventory data via Apache Kafka or Spark Streaming
- Deployment on a multi-node HDFS cluster to remove the single-node memory ceiling
- Addition of a machine learning layer for demand forecasting using Spark MLlib
- Runtime ingestion of `settings.yaml` to fully externalize configuration
- Automated data quality checks using a framework such as Apache Griffin

REFERENCES

- [1] S. Ghemawat, H. Gobioff, and S.-T. Leung, "The Google File System," Proc. 19th ACM Symp. Operating Syst. Principles, 2003.
- [2] J. Dean and S. Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters," Proc. 6th Symp. Operating Syst. Design and Implementation, 2004.
- [3] M. Zaharia et al., "Spark: Cluster Computing with Working Sets," Proc. 2nd USENIX Conf. Hot Topics in Cloud Computing, 2010.
- [4] A. Thusoo et al., "Hive: A Warehousing Solution Over a MapReduce Framework," Proc. VLDB Endowment, vol. 2, no. 2, 2009.
- [5] G. Fourny, *The Big Data Textbook: From Clay Tablets to Data Lakehouses*, 2nd ed., 2024.